Bitwise Operators :

Here's a more detailed breakdown, operator by operator.

AND (&) — result is 1 only if BOTH bits are 1

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

Use case: checking if a specific bit is set (masking).

OR (|) — result is 1 if AT LEAST ONE bit is 1

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

Use case: turning a bit ON without disturbing other bits.

XOR (^) — result is 1 only if the bits are DIFFERENT

a	b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

Use case: toggling a bit, finding the single non-duplicate number in an array, swapping without a temp variable.

NOT (~) — flips every bit (single input, not two)

a	~a
0	1
1	0

Note: in Java, ~a on an int flips ALL 32 bits, so ~5 isn't just "not 5" in a simple sense, it gives a negative number due to two's complement. Use case: rarely used alone, mostly combined with & to clear specific bits.

Now a multi-bit example combining these, using 4-bit numbers for simplicity: a = 6 (0110), b = 3 (0011)

```
a       0 1 1 0
b       0 0 1 1
a & b   0 0 1 0   -> 2
a | b   0 1 1 1   -> 7
a ^ b   0 1 0 1   -> 5
```

Shift operators, with a = 5 (00000101):

```
a << 1  -> 00001010  -> 10   (shifted left by 1, equivalent to *2)
a << 2  -> 00010100  -> 20   (shifted left by 2, equivalent to *4)
a >> 1  -> 00000010  -> 2    (shifted right by 1, equivalent to /2, drops remainder)
```

Comprehension check: using the truth table, what would 5 ^ 5 give you? (Try it bit by bit using the XOR table above.)

==========
==========

Bit masking is using a specially crafted number (the "mask") combined with &, |, or ^ to check, set, or clear specific bits in another number, without touching the rest.

Analogy: think of a mask like a stencil. If you want to paint only certain spots on a wall and leave the rest untouched, you place a stencil (mask) over it. The bitmask does the same thing to bits, it lets you target exactly the bit(s) you care about.

Three common masking operations:

1. Check if a bit is set (read a switch)

```
int mask = 1 << 4;       // targets only bit position 4
if ((num & mask) != 0) { // is that specific bit ON in num?
    // bit 4 is set
}
```

This is exactly what you did in the Sudoku problem — `1 << 4` was your mask for digit 5, and `&` checked if that one bit was on in rowMask.

2. Set a bit (turn a switch ON)

```
int mask = 1 << 4;
num = num | mask;  // turns ON bit 4, leaves all other bits unchanged
```

Also exactly what you did with `rowMask[row] |= bit`.

3. Clear a bit (turn a switch OFF)

```
int mask = ~(1 << 4);  // every bit is 1 EXCEPT position 4
num = num & mask;      // forces bit 4 to 0, leaves everything else unchanged
```

The core idea: a mask is just a number with 1s in the positions you want to affect and 0s everywhere else (or the inverse, for clearing). Combining it with the right operator lets you touch only those specific bits.

Does this connect back to what you did in the Sudoku solution — that `1 << (digit-1)` was literally a mask targeting one specific digit's bit?

==========
==========

Let's trace this with an actual number, say num = 30, and we want to clear (turn OFF) bit position 4.

First, `1 << 4`:

00010000 (only position 4 is ON)

Now `~(1 << 4)` — the NOT operator flips every single bit:

1 << 4: 00010000
~(1 << 4): 11101111 (every bit flipped: 0s become 1s, the 1 becomes 0)

So this mask has a 0 ONLY at position 4, and 1s everywhere else. That's the key: it's the OPPOSITE of a "set" mask.

Now apply `&` with num = 30, which in binary (8-bit) is:

00011110

num: 0 0 0 1 1 1 1 0
mask (~(1<<4)): 1 1 1 0 1 1 1 1
AND result: 0 0 0 0 1 1 1 0

Look at position 4 specifically: num had a 1 there, mask has a 0 there, AND gives 0. Forced OFF. Look at any OTHER position, say position 1: num has 1, mask has 1, AND gives 1. Unchanged. Look at position 0: num has 0, mask has 1, AND gives 0. Also unchanged (stays 0, since it already was 0).

Why this works in general: ANDing with 1 always preserves the original bit (since `x & 1 = x`). ANDing with 0 always forces the result to 0, no matter what the original bit was. So a mask of all-1s-except-one-position-that's-0 clears exactly that one position and leaves everything else exactly as it was.

Comprehension check: if num = 16 (binary `00010000`, only bit 4 is on) and you apply this same clearing mask `~(1<<4)`, what would the result be?